Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed

Kamen Levi
Independent researcher

9 November 2026

Abstract

We study the drafter side of exact speculative decoding for Qwen3.8-Flash-Next, a 125B mixture-of-experts model (48 layers, 512 routed experts per layer, 10 per token) whose experts stream from the internal SSD of one 64 GB Mac ([MAIN]). The engine verifies up to 16 drafted rows per check and keeps the output bit-identical to one-token greedy decoding, so a drafter can change speed but never the output.

We compare two drafters: a 5-layer block drafter that proposes 16 positions in one pass (DFlash2-style), and the model's own multi-token-prediction (MTP) layer, run as a chain, as MTPLX does. Three findings:

- **Training the MTP layer the way it is used**, over its own chain (EAGLE-3's training-time test), raises its code kept tokens per 16-row check from 7.78 to 10.98 (x1.41) [K]. That is above the block drafter's 10.44 on the same answers. Chat moves by only 2-3 %, not significant.
- **The chained MTP head beats the block drafter on chat and agent text** (x1.08-1.32 tokens per check [K]) and loses on code (x0.75). So the engine routes by text type. On 19 fresh chat prompts the switch gave:
  - whole decode 23.1 -> 28.8 tok/s (+24.5 %);
  - whole request 18.6 -> 22.0 tok/s (+18.3 %);
  - first token 1.9 s in both arms;
  - lifetime peak 37.9 GB;
  - every token identical.
- **Most other levers failed.** Of 83 drafter-side items we tried (Appendix A), 41 ended in a KILL (by a pre-registered line, or at the desk or on paper), and 27 ended between lines, parked or as information.

Where we lose is chat. We keep 2.3-2.5 tokens per chat check, and the check's fixed cost is ~75-85 % of a chat check. So a drafter that never missed would still reach only ~105-115 tok/s of chat decode at today's check cost [P]. Against Strata on the same machine, our chat decode is about level, and Strata's first token wins every whole-request number up to 256 generated tokens. We also continued the chat head overnight on the model's own chat features. A memory guard stopped capture at 0.10M tokens, and the head kept x1.014 on held-out chat: a KILL by its own line, at 8 % of its planned data [K].

We also hit a training pitfall. MLX runs fp32 matmuls as TF32 unless told otherwise, which moved our drafter's training losses by 1.3-3.5 % against a float32 port.

Marks: [K] measured, [P] priced (arithmetic or a CPU replay on measured data), [G] judgement. Each number carries a source key; Appendix B maps the keys to files. GB = `10^9` bytes of the lifetime peak footprint (ri_lifetime_max_phys_footprint, as the run reports print it); the 45 GB line is 41.9 GiB, and a memory line is the engine's own footprint on a 64 GB machine, not an OS limit; a 7.45 GB expert tier held as purgeable memory is not counted in it ([MAIN] section 2).

1. Setting

The model and its MTP layer.

- The model's MTP layer is one transformer layer with its own 512 experts (2.61 B parameters, [MAIN] section 3.1).
- It reads the trunk's widened hyper-connection stream (4 x 2560 = 10,240 wide) at position u plus the embedding of token u+1, and predicts token u+2.
- It shares the model's 248,320-token output layer.

- Chained, step j feeds the layer its own output state and its own previous token. The layer was trained for one step, so the stock chain decays: per-depth rate 0.98 at depth 1, 0.70 by depth 15 on code [K, R:'D-22 b3 REAL-RUNTIME RESULT'].

The engine ([MAIN], [KERNELS]) drafts up to 15 tokens per check and verifies them in one pass of 16 rows. Kept per check = accepted drafts + the bonus token. The check is exact to the engine's reference, so drafts never change the output; every live arm below reports token equality across arms.

Drafter A, the block drafter (L1-20).

- 5 layers, 1.21 GB, DFlash2-style: one pass drafts 16 positions from 5 taps of the trunk.
- Fine-tuned from Inco AI's public DFlash2 drafter for this model, incoai/Qwen3.8-Flash-Next-DFlash2 (Hugging Face; revision a22a174 of 19 Oct 2026; Apache-2.0).
- Recipe: forward KL to the model's top-32, block 32, prediction mask 0.5, lr 2e-5, on 99 windows of the model's own code and math answers [K, R:'P-CAP RESULT'; R:'E1 COVERAGE RESULT'].
- It drafts code, math and agent text in the shipped engine.

Drafter B, the MTP chain.

- The native MTP layer chained to K15.
- Cut where the running product of its top-1 probabilities falls below 0.3, with early exit.
- Run through MTPLX by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx), which first ran this model's own MTP head as a drafter on Apple Silicon. Our engine builds on its model code for this architecture.
- It drafts chat in the shipped engine.

Two kinds of measurement.

- **Teacher-forced walks:** the drafter at every position, or at the engine loop's own anchors, of fixed answers of the model. They give exact kept counts for that text, with no speed.
- **Live A/Bs** in the engine. Speeds follow the shared definitions ([MAIN]): whole decode, whole request, first token (median), lifetime peak, pooled over the set.
- The live drafter A/Bs ran on engine versions of 31 Oct - 2 Nov, before the current reference (NAX6-N). Their exactness is token equality across arms, plus equality to that version's own reference where the rows existed.

2. How we measured, and what went wrong in measuring

- **Lines first.** Every screen had a PASS / KILL line written before its run. The verdicts below are those lines' own.
- **Arm order.** On an SSD-streamed model, back-to-back arms on one prompt favour the second arm: it verifies on the experts the first arm just loaded. We measured the factor at x1.12-1.21 per type [P, R:'D-23c' CORRECTION']. One of our PASS reads (+10.6 %) fell to about +8.7 % after the correction, so it was withdrawn. From then on each arm ran its own pass over the set.
- **Retrain noise.** Retraining the block drafter with another seed moves its code kept by 0.16 (9.57 -> 9.73) and math by 0.01 [K, R:'SEED: L1-20 retrain noise']. Single-run code deltas under ~0.3 can't be told from noise. That turned one of our results (+0.11) into "no effect".
- **In-sample prompts**, caught twice:
  - the MTP head's chat windows included 3 of the IFEval prompts of a live A/B (R:'D-23c' CORRECTION');
  - the planned on-policy eval set for the overnight head was inside the old head's training data, and was replaced before the run (R:'F2A night', amendment 1).
- **Alignment.** Four cross-drafter read-outs read the MTP rows one position late. They were corrected the same day, and one lead was withdrawn (R:'ALIGNMENT FIX').
- **TF32 in MLX** (section 3.8).

3. Results

3.1 Training the native MTP head the way it chains

The first attempt failed.

- It trained each depth DeepSeek-style, over a fresh cache of the previous depth's outputs, on 15 agent transcripts.
- Agentic tokens per 16-row pass fell 4.04 -> 3.85 (-4.6 %, 90 % CI -8.4 .. -0.6), and general text -7.5 % [K, R:'The 125B MTP head fine-tune'].
- Depth 1, where training equals inference, rose (0.846 -> 0.858). Depths 2-4 fell, although their training loss fell ~45 %. At inference the chain attends to the true-pair history plus its own entries, not to a fresh cache of its outputs.
- The same read-out showed that putting a tuned head back into the stock format (4 bits per weight) keeps only 12-23 % of the change in those layers. 8-bit with group 64 keeps 99.9 % for ~29 MB more, so tuned heads ship at 8 bits or in bf16 [K, same section].
- Limit: this run's raw read-out files are not in our data tree; the report section holds its numbers.

D-22: training-time test inside the head's attention (EAGLE-3's idea, applied to the native layer).

- Depth j of anchor s takes the head's own depth-(j-1) output and token s+j. Its query sits at s+j-1, over keys = the true pairs up to s plus that anchor's own chain entries: exactly the MTP cache at inference.
- Loss: KL to the model's top-32 on assistant targets.
- Trained in fp32: the head's input projections, attention, router and shared expert. Frozen: its experts, hyper-connection mixers, norms and the output layer. 69.1M trainable parameters [K, R:'D-22 MTPTTT: train the native MTP head'; R:'F2A night', smoke].
- Results in MTPLX's own runtime on 32 held-out prompts, every answer token-identical to the stock-head run:

head	data	code K15 (8 answers)	gsm K15	IFEval K15	multilingual K15	source
stock	-	7.78	6.07	2.43	2.92	R:'D-22 b3 REAL-RUNTIME RESULT'
base (step 76)	99 code + math windows, depths 1-8	9.83 (x1.263, CI 1.185-1.366)	x1.048	-	-	R:'D-22 REAL-RUNTIME RESULT'; TL:data/ d22r_base76_vs_ stock.txt
b3	+ 73 IFEval windows, depths 1-15, one cosine epoch	**10.98** (x1.411, CI 1.287-1.565)	6.86 (x1.131)	2.49 (x1.024, n.s.)	3.00 (x1.028, n.s.)	R:'D-22 b3 REAL-RUNTIME RESULT'; TL:data/ d22r_b3_vs_stock.txt
b4	twice the chat windows (147)	10.65 (x1.368)	7.04 (x1.161)	2.51 (x1.031, n.s.)	3.03 (x1.038, CI 1.013-1.060)	R:'D-22 b4 REAL-RUNTIME RESULT'; TL:data/ d22r_b4_vs_stock.txt
block drafter (L1-20), same answers	-	10.44	6.60	2.24	2.22	R:'D-17 RESULT'

All rows are [K], tokens per 16-row pass (accepted + 1).

- **On code the trained chain becomes target-class:** ~0.97 per depth at every depth 1-15, against stock 0.98 -> 0.70. This took 172 training steps on one GPU [K].
- **Chat barely moves.** Its wall is depth-1 accuracy: IFEval 0.72, multilingual 0.80, code 0.98 [K, R:'D-22 b4 REAL-RUNTIME RESULT']. Twice the chat windows added ~1 %.
- **Stopping each anchor's loss at its chain's first failure (b5, AUF)** gave +0-3 %, significant on math only (MGSM x1.030, CI 1.018-1.042) [K, R:'D-22 b5 (Spec-AUF) RESULT']. b5 became the all-round head that ships for chat.
- **Under the engine's adaptive-width rule the tuned head paid nothing on code** (-8.9 % tokens per pass). The rule had been fitted to the stock head's confidences, and the tuned head is less sure where it is right. In a fixed 7-draft chain the same head gave code +12.5 % tok/s [K, R:'D-22 shipped-line A-B-A redo'].
- **At 24 GB it kept +5-8 % per pass at flat speed**, because decode there is SSD-bound [K, same section].
- **A 3-bit trunk under the head does no harm:** kept per 8-row check 4.02 -> 4.09 [K, R:'MTP3BIT'].

3.2 One drafter per text type

Teacher-forced, same answers [K, R:'D-17 RESULT']. Ratio of tokens per pass, MTP K15 / block drafter (90 % CI):

- code 0.746 (0.703-0.791);
- gsm 0.919;
- IFEval 1.084 (1.032-1.176);
- multilingual 1.315 (1.184-1.464);
- agent 1.295 (1.208-1.372).

A per-pass oracle over the two drafters would add +11 % on code and +43 % on agent text. A cheap router on the model's bonus-token probability got none of it.

Live, in four steps. All arms token-identical.

step	engine (date)	arms	whole decode	whole request	first token (median)	peak	verdict	source
D-20b	REF2b, no confidence cut, C 224 (1 Nov), 17 new prompts	block drafter / switch (MTP K7 on chat, math, agent)	15.95 / 20.30 (+27.3 %); chat 11.38 / 15.07	7.11 / 6.42; chat 9.18 / 11.61 (+26.5 %)	3.94 / 3.94 s	not logged	PASS	R:'D-20b SWITCH45 confirmation'; R:'PAPER SPEED'
D-23	the next default: the block drafter verifies only rows with running confidence >= 0.3 (1 Nov)	block drafter / the same switch	29.43 / 28.65 (-2.7 %)	7.09 / 7.04	3.80 / 3.82 s	42.4 GB	KILL	R:'D-23 SWITCHDEF RESULT'; R:'PAPER SPEED'
D-23c'	the same default, MTP K15 + the 0.3 cut + early exit, C 192, 18 prompts, arms back to back	block drafter / switch	+10.6 % by the letter, about +8.7 % order-corrected [P]	-	-	-	BETWEEN (a PASS withdrawn)	R:'D-23c' CORRECTION'
D-23d	the same, 23 fresh prompts, each arm its own pass (2 Nov)	block drafter / MTP on chat	chat 23.11 / 28.78 (+24.5 %; geo x1.223, 19 of 19 faster)	chat 18.56 / 21.95 (+18.3 %)	1.91 / 1.88 s	37.9 GB	PASS: chat ships on the MTP head	R:'D-23d CHATSWITCH RESULT'; R:'PAPER SPEED'; TL:data/d23d_read.txt

All speed cells are [K], tok/s, from the runs' own rows (R:'PAPER SPEED', TL:data/paper_speed.txt).

Reading.

- **The switch survived the next default only on chat.** That default made the block drafter verify fewer rows (3.9 per chat check), which the fixed 8-row MTP arm could not match. The MTP chain still kept more on chat (3.07 vs 2.71 per check): the loss was the verify width, not the head.
- **Under the same 0.3 cut, the MTP chain wins chat again.** The win is largest where the block drafter is weakest, in languages far from its code and math data. Kept per check on the 16 multilingual prompts went 2.09 -> 2.65. Gains ranged from +5-10 % (French) to +49-50 % (Bulgarian).
- **Agent text stays on the block drafter.** The MTP chain keeps far more per check there (4.80 -> 8.20). But its longer chains verify more rows, which touch more cold experts, so speed was flat.
- **Whole request barely sees the drafter on whole sets.** Long agent contexts took 62-89 s to prefill (median first token), and the prefill doesn't use the drafter. On short chat prompts, three quarters of the decode gain reaches whole request.

3.3 The block drafter: data, capacity, targets

All rows are teacher-forced walks, kept per 16-row check on held-out answers [K].

lever	code	math	verdict	source
data: 1/4, 1/2, all of the base windows	9.37 / 9.46 / 9.57	6.65 / 6.82 / 7.00	slope ~+0.13 per doubling (code)	R:'D-09 RESULT'

lever	code	math	verdict	source
+ the model's own MBPP answers, x2 / x4	9.79 / 9.85	6.99 / 7.06	PASS by its line, flattening; 5-point slope code +0.130, math +0.100 per doubling	R:'E2 / D2 RESULT (x2 + x4)'; TL:data/e2_read.txt
retrain with another seed	9.73	6.99	noise: 0.16 / 0.01	R:'SEED: L1-20 retrain noise'
twice the layers (10, 2.09 GB)	9.75	6.92	inside noise	R:'P-CAP RESULT'
off-policy code (public answers prefilled through the model)	9.68	-	no effect after SEED	R:'P-OFF 60K RESULT'

Killed by their own lines (Appendix A3 has the sources):

- mode-seeking targets (sharpened or total variation, x0.991-0.999);
- a little or ~11x more of the model's own text per language (multilingual kept 2.07 -> 2.06; 2.06 -> 2.05);
- the drafter borrowing the model's own experts;
- loss only at uncertain positions (accuracy there 0.8428 -> 0.8351);
- the MTP head's input stream fed to the block drafter;
- early-layer taps only;
- a second drafter trained to be wrong elsewhere (q = P(wrong given the first is wrong) 0.890-0.893 vs 0.901 for a plain twin);
- two block drafters, the more confident per depth (x1.026).

Reading. The block drafter's gaps are not cheap to close. More of the model's own code is the only lever that held, at ~+0.13 kept per doubling, and code >= 13 kept would need ~23 more doublings [P, R:'E2 / D2 RESULT (x2 + x4)'].

3.4 Combining drafters and picking candidates

- **The room is large.**
  - A perfect top-3 pick among the block drafter's 16 candidates keeps 11.57 on code (x1.209), all 16 12.37 (x1.292) [K oracle, R:'RERANK-0 RESULT'].
  - At the first miss, the model's token is the drafter's rank 2 in 54.7 % of checks.
  - At the model's mid-confidence rows its token is in the drafter's top-3 at 0.888 [K, R:'KWALL (a) RESULT'].
- **Nothing we built captured it.**
  - A per-depth pick between the two drafters keeps x1.073 on code. The MTP chain's ~33 ms per cycle costs more than that, so it is a KILL on paper [K kept, P speed, R:'MTPPICK aligned'].
  - A learned certificate head on the drafter's features: KILL.
  - The MTP head as a second opinion in a learned stop head: KILL (code -1.1 %).
  - Past-only bandit switches: KILL.
  - A per-type confidence cut: within 0-2 % of the shipped 0.3.
- **Trees.**
  - Leaf rows at the MTP chain's low-confidence depths price at -7 % .. +0.9 % on every set [P, R:'D-24 MTPTOP3']. A leaf row costs 7.8 ms, and a rescued chat token is worth ~34 ms.
  - Wider trees need a tree verify for the model's linear-attention layers, which we did not build.
  - 32-row checks keep 12.60 on HumanEval+ and 6.10 on MBPP, under their lines of 14 and 7.5 [K, R:'DEPTH32 RESULT'].

3.5 Copying from the context

- **Copying pays on code and ships there.** Overriding the draft with the continuation of the longest context match keeps x1.038 more per check on 24 HumanEval+ answers in the walk [K, R:'MATCH RESULT']. In the engine, on 12 HumanEval, it keeps x1.08 (geo) / x1.11 (pooled: 11.16 vs 10.01) more per check [K, KS], and whole decode moves x1.077, pooled (125.2 vs 116.2 tok/s) [K, BM:data/r57]. HumanEval answers copy their prompt's docstring, so this flatters: the same drafter keeps 5.94 per check on MBPP against 9.57 on HumanEval+ [K, KS].
- **On chat, copying loses.** Prompt lookup or suffix drafting alone gives x0.44-0.59 tok/s. With the MTP chain it gives x0.89-0.996, and even an oracle choice between the two is x1.012 [P on measured check costs, R:'CHAT150'].
- **Other lookups failed:**
  - the model's own n-gram table as a predictor;
  - computed calculator tokens in math (2.28 % of tokens);

  - schema filling for tool calls (2.2 % of agent first misses fillable);
  - a reduced draft vocabulary: KILL (Appendix A5).

3.6 What a draft step costs

- **The output layer dominates the step.** One MTP draft step takes ~2.2 ms; the 248,320-token output layer is 1.47 ms of it. That runs at 242 GB/s, 85-94 % of the Mac's streaming bandwidth [K, R:'HEADBENCH']. Measured in the engine's own path, a drafted chat row costs 2.59 ms [K fit, R:'CHAT150'].
- **The output layer can't be cut.** At 3 bits it loses 5 % of kept on IFEval, at 2 bits 14 % [K, R:'D1: the chat draft step'].
- **The head's trained parts at 4 bits** are neutral on the exact config: whole decode 36.6 -> 36.9 tok/s, 12 / 12 equal to one-token decoding. On the lossy config they lose: whole decode 43.80 -> 40.22 tok/s (-8.2 %), draft per check 7.9 -> 13.4 ms pooled (8.7 -> 14.9 as the mean over prompts), lifetime peak 42.37 vs 42.28 GB [K, R:'D1 v1b in the engine'; BM:data/chs/chs4_report.txt]. The draft step stays in bf16.
  - Limit: whole request and first token were not in that run's per-arm report.
- **Getting the draft off the critical path.**
  - Pre-drafting the next block during the check is exact: 281 of 281 pre-drafts were identical [K, R:'DA-1 RESULT'].
  - But it pays only with ~20-33 ms of idle GPU per check, and today's 45 GB check leaves <= ~2.5 ms [P, R:'DA-1 re-priced'].
  - Drafting on the CPU: the output layer alone takes 20.0 ms in fp32, 38.7 ms beside a busy GPU, against 1.47 ms on the GPU: KILL [K, R:'CPUMTP'; TL:data/cpu_head_bench.txt].
  - The MTP layer as a one-pass block drafter can't work. In one pass, rows 2-15 have no known token and no chain state: KILL by analysis [R:'MTP-BLOCK'].

3.7 Chat: the check-cost wall

- **Chat's check cost, fitted on the engine's own per-check records** (12 IFEval, 45 GB) [K, R:'CHAT150']:
  - exact config: 47.0 ms + 3.25 ms per verified row + 2.59 ms per drafted row. Estimator: least squares of the cycle wall minus its measured draft time, on rows, over checks >= 2 (TL:sticky/chat_ngram.py). A plain least-squares fit of the check time alone on the same checks gives 44.9 + 4.00 ms per row (the estimator of our independent re-derivation).
  - lossy config: 33.1 ms + 4.35 ms per row;
  - the fixed part is ~75-85 % of a chat check at 2-4 rows under either fit.
- **A drafter that never missed** would reach ~105-115 tok/s at 16 rows with the serial MTP draft (the two fits; both extrapolate from checks of mostly 2-4 rows), or ~135-150 with a one-pass draft [P]. 150 tok/s of chat decode needs the check's fixed part cut ~2.5x and a head ~4x faster.
- **Chat acceptance itself.** The shipped head keeps 2.34 (IFEval) / 2.54 (multilingual) per check [K replay]. The levers that remain on the drafter side are small:
  - a re-tuned cut: x1.015;
  - a 2x / 4x faster head: x1.095 / x1.138;
  - tree leaves, which need a tree verify: x1.009-1.044.
- **F2a, an EAGLE-3-style continuation of the chat head** on the model's own features. Text: UltraChat-200k + Aya; labels: the model's own top-32; the D-22 trainer, 2 epochs, from b5.
  - Before the run it was priced at x1.1-1.3 kept on IFEval [G, R:'F2A feasibility'].
  - Measured [K, R:'F2A night RESULT'; TL:data/f2a/f2a_eval.txt], kept per check, b5 -> F2a:
    - held-out IFEval 2.301 -> 2.334 (x1.014);
    - held-out multilingual 2.507 -> 2.540 (x1.013);
    - on-policy IFEval (24 fresh greedy answers) 2.444 -> 2.450 (x1.002).
  - KILL by its pre-registered line (held-out < x1.03).
  - Limit: the swap guard stopped capture after 27 min at 0.10M tokens, 8 % of the planned ~1.3M. So this kills F2a at 0.1M tokens, not the data-rich plan.
  - Capture ran at 76-80 tok/s [K]. 1.3M tokens therefore need ~4.6 h of quiet machine [P].

3.8 TF32 in MLX

- **What happened.** We ported the block drafter's MLX trainer to PyTorch for free cloud GPUs. The port matched MLX's step-1 loss to 1e-4 but sat 1.5-3.8 % above the logged losses at steps 2-5.
- **The cause** [K, R:'PORT steps FAIL: ROOT CAUSE FOUND']:
  - MLX 0.32.2 runs fp32 matmuls as TF32 unless MLX_ENABLE_TF32=0 (its utils.h default).
  - fp32 matmul error against float64 is 4.15e-4 by default, 6.5e-7 with TF32 off.
  - A 1e-5 relative weight change moves the output ~51x more than it should.
- **The check.** With TF32 off, MLX's own trainer matches the port within 1.53e-3. TF32 on vs off moves the loss -1.3 .. -3.5 % at steps 2-5 [K, R:'PORT full-scale confirmation'].
- **Consequence.** Every "fp32 master weights" MLX run of ours trained under TF32. The logs are deterministic but are not float32 arithmetic.
- Prior reports: MLX documents the default (its docs, "Environment variables": MLX_ENABLE_TF32, "The default is 1"; and "Precision"), and MLX issue #4595 (29 Sep 2026) reports its cost for fp32 convolutions on the M5 since 0.32. We claim only that we hit it.

4. Early headlines that were wrong

The set's withdrawn headlines ([MAIN]): "12 GB, 40.8 tok/s 'exact'" (a FAST arm; withdrawn); "45 GB FAST stack, 157.9 tok/s" (included CD16, which failed MBPP; withdrawn); "180.3 tok/s" was the registered steady metric, which is 142.8 on whole decode, pooled (metric corrected 7 Nov; the correction note's 142.1 is a geo mean). On the drafter track:

- **"The MTP chain doesn't beat the block drafter on chat"** (a desk check on depths 1-3). Overturned by measurement: x1.08 IFEval, x1.32 multilingual (R:'D-17 RESULT').
- **"The tuned head is +10-33 % on the shipped line."** It always ran second, on experts the stock arm had just loaded (R:'D-22 on the shipped 45 GB line, first A/B').
- **"The MTP chat switch passes, +10.6 %."** Withdrawn the same morning for arm order and in-sample prompts; BETWEEN (R:'D-23c' CORRECTION').
- **"The two drafters are wrong at different places (q 0.301)."** An off-by-one read. Aligned, q is 0.436, and four read-outs were corrected (R:'ALIGNMENT FIX').
- **"Off-policy data adds +0.11 kept."** No effect, once retrain noise (0.16) was measured (R:'SEED').
- **"One epoch over 20M tokens of cloud training in ~2-3 h."** About 15x too low: ~35-45 h on 2 x T4 [P, TL:notes/PORT_PLAN.md].
- **A wrong claim in our own write-up** ("the chat head was trained on code and math only"). 5 of 8 IFEval keys of the next test were in its training. Caught before that test ran.
- **CD16**, keeping the drafter's comment text unchecked: x1.20 kept on HumanEval, withdrawn after MBPP's tests failed ([MAIN] section 4; KS).

5. Where we lose

- **Against Strata.** Strata (MIT; github.com/Niko1221/Strata) ran on the same machine and the same 48 prompts. Our decode is ahead on code and math and about level on chat, our exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (about 3-4.6 s). So up to 256 generated tokens Strata wins or ties every whole-request number. Source: F:2026-11-08_1630_strata-head-to-head ([MAIN] has the table).
- **On chat, from the drafter's side:**
  - we keep 2.3-2.5 tokens per check;
  - the MTP head's next-token accuracy on IFEval is 0.73-0.76, against 0.98 on code;
  - Strata's own MTP drafting accepts 87 % of drafted tokens and gives it +53 % decode (25.3 -> 38.7 tok/s pooled) [K, our run of Strata, same source]. Its draft length per step is not in our records.
- **Data.** Capturing the model's own features runs at 76-80 tok/s here [K], so a night gives ~1-1.3M tokens at best; our one overnight run got 0.10M before a memory guard stopped it. EAGLE-3 trains its chat heads on far more: ShareGPT and UltraChat-200K, about 68K and 464K conversations, with the responses regenerated by the target model (arXiv 2503.01840; it gives no token count; self-reported).
- **Code gains are HumanEval-flavoured.** MBPP keeps 5.94 per check against HumanEval+'s 9.57 with the same drafter (KS).

6. Related work

Every entry below was checked against its source by the set's related-work check (Mon 9 Nov; arXiv ids against their titles). What each does, and what we took:

- **Speculative decoding** (Leviathan et al. 2023, arXiv 2211.17192): the exact accept rule our check implements.
- **MTPLX** by Youssof Altoukhi (Apache-2.0; github.com/youssofal/mtplx): ran this model's MTP head as a chained drafter on Apple Silicon. We use its model code and its chain; our trained heads load into it.
- **DeepSeek-V3's MTP and FastMTP** (2412.19437, 2509.18362): train MTP modules for multi-step drafting. FastMTP's recursion training is the closest prior to D-22.
- **EAGLE, EAGLE-2, EAGLE-3** (2401.15077, 2406.16858, 2503.01840): feature-level drafters. EAGLE-3's training-time test is D-22's training scheme, applied inside the native layer's attention.
- **Training the chain the way it runs:** AngelSpec (Liu et al., 2607.25852), one training framework for MTP and block drafters; and Badarayani et al., "Match the Distribution, Not the Compute: Post-Training Multi-Token Prediction Heads" (2610.00888), which post-trains chained MTP heads on a frozen model's own outputs.
- **DFlash** (2602.06036), its DFlash2 drafters, and DFlare (2606.02091): block drafters. Our block drafter is a fine-tuned DFlash2.
- **Medusa, SpecInfer, Sequoia, SpecExec, EAGLE-2's dynamic trees** (2401.10774, 2305.09781, 2402.12374, 2406.02532, 2406.16858): token trees. Our priced trees use their costs.
- **FR-Spec** (2502.14856): reduced draft vocabularies (killed here).
- **Prompt lookup, SuffixDecoding, SAM-Decoding** (SuffixDecoding 2411.04975; SAM-Decoding 2411.10666): copying (MATCH ships on code).
- **ToolSpec** (2604.13519): schema-guided tool drafting (killed here).
- **SpecDec++ and DISCO** (2405.19715, 2405.04304): learned acceptance and stop heads (killed here).
- **SP-MoE and Specter** (2510.10302, 2511.14102): expert prefetch inside speculative decoding (Appendix A7).
- **Ghidorah, DuoDecoding, PEARL** (2505.23219, 2503.00784, 2408.11850): parallel or CPU-side drafting (killed here).
- **WhiFlash** (2606.07710): switches between an autoregressive and a diffusion drafter token by token; our switch picks one drafter per task ([MAIN] cites it).
- **Strata** (MIT): MTP drafting in a streamed-expert engine (section 5).

7. What we left on the table

Measured room we did not use:

- **A better pick.** x1.209 (top-3) / x1.292 (all 16) on code with a perfect pick among the block drafter's candidates [K oracle]. The model's token is in the drafter's top-3 at 0.888 of mid rows [K].
- **A per-pass choice of drafter.** Oracle: +11.3 % code, +43.1 % agent, +18.6 % IFEval, +43.4 % multilingual [K, R:'D-17 RESULT'].
- **Width on chat.** An oracle width of 2 / 3 at every depth keeps 3.6 / 4.05 tokens per pass against 2.3-2.5 cut [K, R:'D-24 MTPTOP3']. It needs a cheap tree verify for the linear-attention layers.
- **Draft-ahead.** Exact (281 / 281); it pays once the check leaves >= ~20-33 ms of GPU idle [K / P].
- **Data.** +0.13 kept per doubling of the model's own code answers, still rising [K]. The data-rich chat-head run (~1.3M tokens, ~4.6 h of capture [P]) and its data-scaling arm did not run before the release.
- **Lossy acceptance on chat.** Near-tie acceptance gives +8 .. +23 % tokens per pass at rho 0.25 on stored chat answers [K replay, lossy, ungated, R:'R-61N RESULT']. The relaxed rules that passed a quality gate are in [EVAL].
- **The chat check's fixed part.** 75-85 % of a chat check [K]: not drafter room, but the drafter's ceiling.

8. Reproducibility

- Training: TL:sticky/d22_ttt.py, the D-22 heads; TL:sticky/f2a_*.py, the overnight chat head; runners in TL:turns/.
- Walks and replays: TL:sticky/d02b_walk.py, d17_*.py, d24_*.py, chat_ngram.py, paper_speed.py.

- Heads: TL:ckpt/d22_b5.safetensors (the shipped chat head; not in the public snapshot) and its training state.
- Raw data: TL:data/.
- The engine loads the chat head named by R57_MTP_HEAD (default b5) and checks its md5 [K, BM:scripts/r57_screen_fast.py].
- Software: MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 ([KERNELS]); 0.32.4 changes tokens against our exact references.
- Code: GPL-3 with the engine ([RELEASE]).

Appendix A. Every drafter-track item, its verdict and its source

Verdicts come from each item's own line, written before its run, unless the row says desk / on paper (a replay or a cost model, no live run). "R:" = a report section found by the words given. Rows marked (log) have no report section; their verdict is from the session log and carries no number here.

A1. Training the native MTP head

item	what	verdict	source
mtp125n_d4	DeepSeek-style depth training (each depth on a fresh cache), 15 agent transcripts	KILL	R:'The 125B MTP head fine-tune'
re-store at 4 bits	put a tuned head back into the stock format (4 bits per weight)	KILL (store tuned heads at 8-bit or bf16)	R:'The 125B MTP head fine-tune'
D-22 base	training-time test inside the head's attention (EAGLE-3 style), depths 1-8, code + math windows	PASS	R:'D-22 REAL-RUNTIME RESULT'
D-22 2 epochs	the same, longer	KILL	R:'D-22 interim'
D-22 b3	depths 1-15 + 73 IFEval windows + one cosine epoch	PASS	R:'D-22 b3 REAL-RUNTIME RESULT'
b3 in the shipped adaptive-width line	the tuned head inside a width rule fitted to the stock head	KILL as is (pays in a fixed chain)	R:'D-22 shipped-line A-B-A redo'
b3 at 24 GB	the tuned head where decode is SSD-bound	KILL for speed (kept +5-8 %)	R:'D-22 shipped-line A-B-A redo'
D-22 b4	twice the chat windows	BETWEEN	R:'D-22 b4 REAL-RUNTIME RESULT'
D-22 b5	loss stopped at each chain's first failure (AUF)	BETWEEN (kept as the all-round head)	R:'D-22 b5 (Spec-AUF) RESULT'
MTP3BIT	the head over a 3-bit trunk	no harm	R:'MTP3BIT'
F2a	the head continued on the model's own chat features (0.10M tokens; 1.3M planned)	KILL by its line, at 8 % of the planned data	R:'F2A night RESULT'

A2. Which drafter for which text

item	what	verdict	source
D-12	desk check: MTP chain vs block drafter on chat, depths 1-3	KILL at the desk, overturned by D-17	(log)
D-17	the MTP chain to K15 vs the block drafter, same answers	PASS (chat / agent)	R:'D-17 RESULT'
D-17 router	MTP or block per pass from the model's bonus-token p	KILL	R:'D-17 RESULT'
D-17 oracle / 2-branch tree	both chains in one check	PARKED (no selector; rows cost bytes)	R:'D-17 RESULT'
D-18	the MTP head's first guess revealed to the block drafter	KILL (desk)	R:'D-18 MTP1'
D-19	past-only switch rules (bandits) vs a text-type rule	KILL; the type rule PASS	R:'D-19 RESULT'
D-05	MTP drafts the near depths, the block drafter the rest	BETWEEN (not worth a build)	R:'D-05 MTPHYB + D-08 CALC walk'
D-20	type switch live, REF2b, C 160	PASS	R:'D-20 SWITCH45 RESULT'
D-20b	the same at C 224 on new prompts	PASS	R:'D-20b SWITCH45 confirmation'
D-23	the same switch on the newer default (confidence cut on the block drafter)	KILL	R:'D-23 SWITCHDEF RESULT'

item	what	verdict	source
D-23b	CPU replay: the MTP chain under the same cut	PARKED, measured by D-23c'	R:'D-23b MTP TRUNC'
D-23c'	MTP K15 + cut + early exit, live	BETWEEN (a PASS withdrawn: arm order + in-sample prompts)	R:'D-23c' CORRECTION'
D-23d	chat switch on 23 fresh prompts, arms in separate passes	PASS (shipped: chat on the MTP head)	R:'D-23d CHATSWITCH RESULT'
D-23e	the chat switch at C 224	DROPPED before any run	R:'D-23e DROPPED'
D-23x	the first check after a long prefill, read-traced	measured (a cold pool after the prefill)	R:'D-23x RESULT'

A3. The block drafter: data, capacity, targets, inputs

item	what	verdict	source
D-02a	the block drafter on chat; how sure the model is at its first misses	measurement	R:'D-02a RESULT'
D-09	1/4, 1/2, all of the base data	BETWEEN (a slow lever)	R:'D-09 RESULT'
D-10	soft KD targets	withdrawn: the recipe already uses forward KL to the model's top-32	(log)
D-13	mode-seeking targets (sharpened, total variation)	KILL	R:'D-13 RESULT'
D-14 C	a longer drafter window for agent text	KILL (stored data)	R:'D-14 probes'
D-15a	a little of the model's own text per language	KILL	R:'D-15a RESULT'
D-16	the MTP head's input (the widened hyper-connection stream) fed to the block drafter	KILL at the rapid protocol (may under-read)	R:'D-16 RESULT'
D-21	a second refinement step per block	KILL (measured earlier)	(log)
E1	~18K of the model's own answer tokens in each of 3 languages	KILL	R:'E1 COVERAGE RESULT'
E5	the block drafter borrows the model's own experts	KILL	R:'E5 DFlash-X RESULT'
HPD-1	2 more epochs with the loss only at uncertain positions	KILL	R:'HPD-1 RESULT'
E2 / D2	one and two more doublings of code data (the model's own MBPP answers)	PASS (a flattening slope)	R:'E2 / D2 RESULT (x2 + x4)'
EARLYTAP	read only early layers' features (so drafting can overlap the check)	KILL	R:'EARLYTAP RESULT'
DIVERSE2	a second drafter trained to be wrong elsewhere	KILL	R:'DIVERSE2 RESULT'
DUALDRAFT	two block drafters, the more confident per depth	KILL	R:'DUALDRAFT RESULT'
P-CAP	twice the layers (block expansion)	BETWEEN (inside retrain noise)	R:'P-CAP RESULT'
P-OFF	off-policy code answers prefilled through the model	ALIVE as data, no effect on kept	R:'P-OFF diagnostic RESULT'; R:'P-OFF 60K RESULT'
SEED	the base recipe rerun with another seed	measurement (retrain noise)	R:'SEED'
#20 BIGDRAFT	a block drafter 4-8x larger	PARKED (desk; needs paid compute)	R:'#20 BIGDRAFT'
TTS1	test-time LoRA on the block drafter after every check	KILL	R:'TTS1'
D7B-1	the block drafter at 4 bits vs bf16	information	R:'D7B-1'
MTP-BLOCK	the MTP layer as a one-pass block drafter	KILL by analysis, no run	R:'MTP-BLOCK'

A4. Combining drafters, picking candidates, trees, cuts

item	what	verdict	source
D-24 leaves	rank-1 / rank-2 leaf rows at the MTP chain's low-confidence depths	KILL (cost model on measured ranks)	R:'D-24 MTPTOP3'
D-24 K15 + cut	K15 vs K7 under the 0.3 cut	PASS on paper (then D-23c')	R:'D-24 MTPTOP3'
D-24 cut	retune the 0.3 cut for the MTP chain	KILL (keep 0.3)	R:'D-24 MTPTOP3'
R-61N near-tie	accept the drafter's token if its p >= rho x the model's top-1 p (lossy)	BETWEEN (ungated)	R:'R-61N RESULT'
R-61N top-3	a 2-3 way branch at the block drafter's first misses	PARKED (a lead)	R:'R-61N RESULT'

item	what	verdict	source
K-HEAD	a learned certificate head on the drafter's features	KILL	R:'K-HEAD RESULT'
KWALL (a)	is the model's token among the drafter's candidates at mid rows?	PASS (a selector problem)	R:'KWALL (a) RESULT'
KWALL (b)	the MTP head's depth-1 accuracy by the model's confidence class	BETWEEN	R:'KWALL (b) RESULT'
MTPPICK	the more confident of the two drafters per depth, one chain	KILL on paper (aligned)	R:'MTPPICK aligned'
MTPSCREEN	the same pick with a screened (cheaper) MTP head	PASS by its line, but the pick is small	R:'MTPSCREEN RESULT'
TOP3-USE	trees of top-3, a learned picker, or MTPPICK, priced	KILL on paper	R:'TOP3-USE'
RERANK-0	the ceiling of a perfect pick among the 16 candidates	PASS at its edge (a ceiling, not a lever)	R:'RERANK-0 RESULT'
SELGAP	the block drafter's own selector on vs off	OPEN, not queued	R:'SELGAP'
CUTSWEEP	a per-type cut for the block drafter	no knob (within 0-2 %)	R:'CUTSWEEP'
DEPTH32	32-row checks with both drafters	KILL	R:'DEPTH32 RESULT'
M-23b	the MTP head as a second opinion in a learned stop head	KILL	R:'M-23b FIRSTMISS-MTP RESULT'
ANCHORCOV	pre-drafting the next block from 3 anchors	KILL by its rule	R:'ANCHORCOV RESULT'
TREE3 + STAGED	aligned top-3 tree data, priced for staged verification	data + price	R:'TREE3 + STAGED'

A5. Copying from the context and other lookups

item	what	verdict	source
D-03	the model's own n-gram table as a next-token predictor	KILL	R:'D-03 NGRAM'
D-08	compute calculator-determined math tokens exactly	KILL	R:'D-05 MTPHYB + D-08 CALC walk'
MATCH	the longest context match overrides the draft	BETWEEN (walk); shipped in the engine's code profile	R:'MATCH RESULT'; KS
COPYGATE	MATCH behind a precision gate	BETWEEN	R:'COPYGATE RESULT'
TOOLSPEC (a)	schema-FSM fill of agent first misses	KILL	R:'TOOLSPEC'
FR-Spec vocab	draft over a reduced vocabulary	KILL (desk)	R:'FR-Spec-style reduced draft vocab'
CHAT150 lookup	prompt lookup / suffix drafting on chat, alone and with the MTP chain	KILL	R:'CHAT150'
T16-HE	comment rows kept when the model ranks them top 3 (lossy, code)	KILL (kept +3 %, cycle +1.9 ms)	R:'T16-HE'

A6. The draft step's cost, and getting it off the critical path

item	what	verdict	source
HEADBENCH	what the MTP step's output layer costs	KILL for a kernel (near bandwidth)	R:'HEADBENCH'
D1	the step's anatomy; output layer at 3 / 2 bits; trained parts at 4 bits; no-sync chain	output-layer cuts KILL; parts at 4 bits small	R:'D1: the chat draft step'
D1 v1 / v1b	the trained parts at 4 bits, in the engine	neutral exact, KILL lossy	R:'D1 v1b in the engine'
CPUMTP	MTP draft steps on the CPU during GPU waits	KILL	R:'CPUMTP'
DA-0 / DA-1	pre-draft the next block during the check	BETWEEN / PASS (exact), but it pays only with idle GPU	R:'DA-0 RESULT'; R:'DA-1 RESULT'
DA-1 re-priced	on the GPU-bound 45 GB check	slower (no idle GPU)	R:'DA-1 re-priced'
DRAFTOFF	what it takes to take drafting off the critical path	estimate	R:'DRAFTOFF estimate'
DRAFT17 / D17-1	a 17th draft position for the bonus row	answered (cost) / information	R:'DRAFT17 RESULT'; R:'D17-1'
CH3	a 2x / 4x faster MTP step, priced on chat	x1.095 / x1.138 [P]	R:'CHAT150'

A7. Adjacent (the SSD prefetch side, run on drafter data)

item	what	verdict	source
ROUTEPRED	a learned next-check expert-route predictor	KILL on paper	R:'ROUTEPRED'
LEAD1CORR	a learned next-layer forecast	information	R:'LEAD1CORR'
S2	a cheap chain one cycle early as a route predictor	PASS by the letter, small in speed	R:'S2'
SAMELAYER-T	a ridge-trained same-layer forecast router	screened, then fit and delivered	R:'PR160 read check + SAMELAYER-T'; R:'SAMELAYER-T FIT RESULT'

A8. Methods

item	what	verdict	source
PORT	a PyTorch port of the block drafter's MLX trainer (for free cloud GPUs)	validated; found MLX's TF32 default	R:'PORT steps FAIL: ROOT CAUSE FOUND'; R:'PORT full-scale confirmation'
DLOG	log the model's own answers from the live exact loop as training data	a tool	R:'E5 DFlash-X' (its DLOG paragraph)
PAPER SPEED	whole request and first token from the live A/Bs' rows	derivation	R:'PAPER SPEED'

Appendix C. Headline numbers and their sources

One row per headline number in the text; every row was re-read from its source on Sun 11-08.

#	result	number	mark	config	source
1	Native MTP head trained with the training-time test (b3), MTP runtime, 8 code answers	tokens per 16-row pass 7.78 -> **10.98** (x1.411, 90 % CI 1.287-1.565); per-depth rate ~0.97 at every depth 1-15 (stock 0.98 -> 0.70); the block drafter on the same answers: 10.44	[K]	drafts only; every answer token-identical to the stock-head run	R:'D-22 b3 REAL-RUNTIME RESULT'; TL:data/d22r_b3_vs_stock.txt
2	Same head (b3), other sets	gsm 6.07 -> 6.86 (x1.131, CI 1.093-1.180); IFEval 2.43 -> 2.49 (x1.024, n.s.); multilingual 2.92 -> 3.00 (x1.028, n.s.)	[K]	same	same
3	Twice the chat windows (b4)	IFEval 2.51 (x1.031, n.s.), multi 3.03 (x1.038, CI 1.013-1.060), code 10.65 (x1.368). Chat's wall is depth-1 accuracy: IFEval 0.72, multi 0.80, code 0.98	[K]	same	R:'D-22 b4 REAL-RUNTIME RESULT'; TL:data/d22r_b4_vs_stock.txt
4	Loss stopped at each chain's first failure (b5, AUF)	+0-3 %, significant on math only (MGSM K15 x1.030, CI 1.018-1.042); code and chat flat	[K] teacher-forced	drafter only	R:'D-22 b5 (Spec-AUF) RESULT'; TL:data/d22_b5_vs_b4.txt
5	The earlier DeepSeek-style tune (each depth on a fresh cache)	agentic tokens per 16-row pass 4.04 -> 3.85 (-4.6 %, 90 % CI -8.4..-0.6); general text -7.5 %; depth 1 rose (0.846 -> 0.858), depths 2-4 fell. KILL	[K]	greedy output token-identical to the stock run	R:'The 125B MTP head fine-tune (exact-safe ...): killed'; raw read-out files not in our data tree (stated limit)

#	result	number	mark	config	source
6	Drafter by text type, teacher-forced, same answers	MTP K15 / block drafter, tokens per pass: code 0.746 (0.703-0.791), gsm 0.919, IFEval 1.084 (1.032-1.176), multi 1.315 (1.184-1.464), agent 1.295 (1.208-1.372)	[K]	drafter only	R:'D-17 RESULT'; TL:data/d17_route.txt, d17_ci.txt
7	The type switch live, no confidence cut (REF2b, C 224, 17 new prompts)	whole decode, whole set 15.95 -> 20.30 tok/s (+27.3 %); pooled per type, block drafter -> MTP K7: chat +32.4 % (11.38 -> 15.07; faster on 6 of 6 prompts), math +19.8 %, agent +42.0 %, code -14.7 % (so code stays on the block drafter). Whole request: chat 9.18 -> 11.61 (+26.5 %); whole set 7.11 vs 6.42, because 3 agent prompts' first tokens (median 62-66 s) dominate and their prefills ran longer in the MTP arm. First token median 3.94 s both. Peak not logged (stated limit)	[K]	exact (all arms' tokens equal)	R:'D-20b SWITCH45 confirmation'; R:'PAPER SPEED'; TL:data/d20_ runs.jsonl, data/paper_ speed.txt
8	The same switch on the next default (block drafter verifies only rows with running confidence >= 0.3)	whole set 29.4 -> 28.6 tok/s (-2.7 %): KILL; peak 42.4 GB. The MTP arm still kept more on chat (3.07 vs 2.71 per check) but verified a fixed 8 rows against 3.9. Whole request 7.09 vs 7.04; first token median 3.80 / 3.82 s	[K]	exact	R:'D-23 SWITCHDEF RESULT'; R:'PAPER SPEED'; TL:data/d23_ runs.jsonl, data/paper_ speed.txt
9	MTP K15 + the same 0.3 cut + early exit (D-23c', 18 prompts, C 192)	+10.6 % whole set by the letter, then BETWEEN after the order correction: about +8.7 % (a model with a constant w per type)	[K] raw, [P] corrected	exact	R:'D-23c' at C 192 FINAL'; R:'D-23c' CORRECTION'; TL:data/d23c_ mtptrunc.txt
10	Chat switch on 23 fresh prompts, each arm its own pass (D-23d, C 192)	chat time-weighted +24.5 %, geo x1.223, faster on 19 of 19; multi +27.4 % (kept per check 2.09 -> 2.65); IFEval +1.4 % (n = 3); agent -0.5 % (stays on the block drafter). Chat whole decode 23.11 -> 28.78 tok/s; whole request 18.56 -> 21.95 (+18.3 %); first token median 1.91 / 1.88 s; lifetime peak 37.9 GB	[K]	exact (every token equal across arms)	R:'D-23d CHATSWITCH RESULT'; R:'PAPER SPEED'; TL:data/d23d_ read.txt, data/paper_ speed.txt, logs/d23d.out
11	Block drafter, data slope	code kept per check 9.37 / 9.46 / 9.57 at 1/4, 1/2, all of the base data; + the model's own MBPP answers x2 9.79, x4 9.85. 5-point slope: code +0.130, math +0.100 per doubling	[K] teacher-forced	drafter only	R:'D-09 RESULT'; R:'E2 / D2 RESULT (x2 + x4)'; TL:data/e2_read.txt

#	result	number	mark	config	source
12	Retrain noise	one reseed: code 9.57 -> 9.73 (0.16), math 7.00 -> 6.99	[K]	drafter only	R:'SEED: L1-20 retrain noise' (its RESULT)
13	Capacity, 10 layers (2.09 GB) vs 5 (1.21 GB)	code 9.57 -> 9.75, math 7.00 -> 6.92: inside retrain noise	[K]	drafter only	R:'P-CAP RESULT'
14	Off-policy code data	+0.11 code: no effect once retrain noise was known	[K]	drafter only	R:'P-OFF 60K RESULT'
15	A perfect pick among the block drafter's candidates (ceiling, not a lever)	code 9.57 -> top-3 11.57 (x1.209), all 16 12.37 (x1.292); at the first miss the model's token is the drafter's rank 2 in 54.7 %	[K] oracle	drafter only	R:'RERANK-0 RESULT'; TL:data/rerank0.txt
16	Per-depth pick between the two drafters (aligned)	code 9.58 -> 10.28 kept (x1.073), but the MTP chain's ~33 ms per cycle costs more: KILL	[K] kept, [P] speed	drafter only	R:'MTPPICK aligned'; TL:data/mtppick_sim.txt
17	Leaf trees on the MTP chain	-7 % .. +0.9 % on every set; a leaf row costs 7.8 ms, a rescued chat token is worth ~34 ms: KILL	[P] on measured ranks	needs a tree verify (not built)	R:'D-24 MTPTOP3'; TL:data/d24_read.txt
18	32-row checks	best realizable arm 12.60 HumanEval+ / 6.10 MBPP kept per check, lines 14 / 7.5: KILL	[K] walk	drafter only	R:'DEPTH32 RESULT'; TL:data/d23h_read.txt
19	Copying the context's continuation over the draft (MATCH, 16 rows)	kept per check: walk x1.038 on 24 HumanEval+; engine x1.08 geo / x1.11 pooled (11.16 vs 10.01; 1.114 unrounded) on 12 HumanEval. Whole decode x1.077 pooled (125.19 vs 116.23 tok/s; geo x1.056). Shipped in the code profile	[K]	the lever keeps exactness; the engine figure is on the lossy code profile	R:'MATCH RESULT'; KS; BM:data/fday, data/sat; BM:data/r57/st{b,bb,s,sb}_224_runs.json
20	Prompt lookup / suffix drafting on chat	alone x0.44-0.59 tok/s, hybrids with the MTP chain x0.89-0.996, an oracle selector x1.012: KILL	[P] replay on measured check costs	exact	R:'CHAT150'; TL:data/chat150_read.txt
21	What one MTP draft step costs	the 248K-vocabulary output layer is 1.47 ms of ~2.2 ms, at 242 GB/s = 85-94 % of the Mac's streaming bandwidth	[K]	-	R:'HEADBENCH'; TL:data/head_bench.txt
22	The trained head's bf16 parts quantized to 4 bits, in the engine (12 IFEval, C 256)	exact config neutral (whole decode 36.61 -> 36.94 tok/s); lossy config KILL twice: v1 draft per check 8.6 -> 14.4 ms pooled (9.1 -> 16.2 mean over prompts); v1b whole decode 43.80 -> 40.22 tok/s (-8.2 %), draft 7.9 -> 13.4 ms pooled (8.7 -> 14.9 mean over prompts); lifetime peak 42.37 (L) vs 42.28 GB (L + v1b)	[K]	exact (12 / 12 equal to one-token decoding) / lossy ungated	R:'D1 v1b in the engine'; BM:data/chs/chs4_report.txt (arms y0, y0p4); whole request and first token not in that run's per-arm report (stated limit)

#	result	number	mark	config	source
23	Chat check cost and the drafter ceiling	exact: 47.0 ms + 3.25 ms per row + 2.59 ms per drafted row (least squares of wall minus draft on rows, checks >= 2; a plain fit of the check time gives 44.9 + 4.00 per row); lossy 33.1 + 4.35 per row; the fixed part is ~75-85 % of a chat check at 2-4 rows. A drafter that never misses: ~105-115 tok/s at 16 rows with the serial draft, ~135-150 with a one-pass draft	[K] fit, [P] ceiling	exact / lossy	R:'CHAT150'; TL:data/chat150_read.txt; BM:data/r57/nie, *nil*_256_checks.json
24	TF32 in MLX 0.32.2	fp32 matmuls run as TF32 by default: error vs float64 4.15e-4 (6.5e-7 with MLX_ENABLE_TF32=0). With it off, MLX's trainer matches our PyTorch port within 1.53e-3; on vs off moves the training loss -1.3..-3.5 % at steps 2-5	[K]	-	R:'PORT steps FAIL: ROOT CAUSE FOUND'; R:'PORT full-scale confirmation'; TL:port/test_mlx_tf32.py, data/port/
25	A 3-bit trunk under the MTP head	kept per 8-row check 4.02 -> 4.09: no harm	[K]	lossy trunk	R:'MTP3BIT'; TL:data/mtp_3bit_vs_4bit.txt
26	Guarded comment-row acceptance on HumanEval (T16)	kept x1.031 (geo); whole decode 173.2 vs 173.2 tok/s: KILL; peak 42.9 GB	[K]	the certified lossy code config (v1' + wired, [EVAL]); the T16 arm itself was never gated	R:'T16-HE' (its RESULT); BM: data commit a03e25d (scripts/t16he_ab.sh 7bc901e)
27	Chat head continued on 0.10M tokens of the model's own chat features (F2a; the swap guard stopped capture at 8 % of plan)	held-out kept x1.014 (IFEval 2.301 -> 2.334, multi 2.507 -> 2.540); on-policy x1.002 (2.444 -> 2.450): KILL	[K]	drafter only	R:'F2A night RESULT'; TL:data/f2a/f2a_eval.txt, capture.log
28	Competitor drafting: Strata's MTP layer on the same model, from our own runs	+53 % decode (25.3 -> 38.7 tok/s pooled, all sets), 87 % of drafted tokens accepted	[K]	its lossy default (Q8_0 hyper-connections)	F:2026-11-08_1630_strata-head-to-head

Appendix B. Sources

Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed.

key	file
R:'*words*'	a section of `TrainLab/REPORT.md`, found by the words of its heading (line numbers move)
TL:*path*	a file under `TrainLab/` (raw data `data/`, scripts `sticky/`, runners `turns/`, notes `notes/`, heads `ckpt/`)
BM:*path*	a file under `BigMoE/` (the engine's own run records and scripts)
KS	`TrainLab/notes/K_SCOREBOARD.md` (the engine's kept-per-check table, from BM:data/fday and BM:data/sat)
F:*slug*	a note in `FableGod/findings/`
[MAIN], [KERNELS], [EVAL], [RELEASE]	the companion papers (see References)

Raw files behind the headline rows:

- TL:data/d22r_base76_vs_stock.txt, d22r_b3_vs_stock.txt, d22r_b4_vs_stock.txt, d22_b5_vs_b4.txt (section 3.1);

- TL:data/d17_route.txt, d17_ci.txt, d20_runs.jsonl, d20b_summary.txt, d23_runs.jsonl, d23_switchdef.txt, d23c_runs.jsonl, d23c_mtptrunc.txt, d23d_read.txt, paper_speed.txt, logs/d23d.out (3.2);
- TL:data/d02b_walk*.txt, e2_read.txt, e15_read.txt, dd_read.txt, dv_read.txt, d13_ratio.txt, d15_read.txt (3.3);
- TL:data/rerank0.txt, kw_read.txt, mtppick_sim.txt, d24_read.txt, d23h_read.txt, cut_sweep.txt, m23b.txt (3.4);
- TL:data/match_read.txt, gate_read.txt, chat150_read.txt (3.5);
- TL:data/head_bench.txt, mtp_step_bench*.txt, d1_variants.txt, da_check.txt, cpu_head_bench.txt; BM:data/chs/chs4_report.txt (3.6);
- TL:data/chat150_read.txt, f2a_headroom.txt, f2a/f2a_eval.txt (3.7);
- TL:port/test_mlx_tf32.py, data/port/ (3.8).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Prachi Badarayani et al. "Match the Distribution, Not the Compute: Post-Training Multi-Token Prediction Heads." arXiv:2610.00888, 2026.
- Tianle Cai et al. "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads." arXiv:2401.10774, 2024.
- Yuxuan Cai et al. "FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction." arXiv:2509.18362, 2025.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Liangkun Chen et al. "SP-MoE: Speculative Decoding and Prefetching for Accelerating MoE-based Model Inference." arXiv:2510.10302, 2025.
- Zhuoming Chen et al. "Sequoia: Scalable, Robust, and Hardware-aware Speculative Decoding." arXiv:2402.12374, 2024.
- DeepSeek-AI. "DeepSeek-V3 Technical Report." arXiv:2412.19437, 2024.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Yuxuan Hu et al. "SAM Decoding: Speculative Decoding via Suffix Automaton." arXiv:2411.10666, 2024.
- Kaixuan Huang, Xudong Guo and Mengdi Wang. "SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths." arXiv:2405.19715, 2024.
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Young D. Kwon et al. "WhiFlash: Accelerating Speculative Decoding with Token-Level Cross-Paradigm Routing." arXiv:2606.07710, 2026.
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- Yuhui Li et al. "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees." arXiv:2406.16858, 2024.
- Yuhui Li et al. "EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test." arXiv:2503.01840, 2025.
- Yuhui Li et al. "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty." arXiv:2401.15077, 2024.
- Hong Liu et al. "AngelSpec: Towards Real-World High Performance Inference with Speculative Decoding." arXiv:2607.25852, 2026.

- Tianyu Liu et al. "PEARL: Parallel Speculative Decoding with Adaptive Draft Length." arXiv:2408.11850, 2024.
- Kai Lv et al. "DuoDecoding: Hardware-aware Heterogeneous Speculative Decoding with Dynamic Multi-Sequence Drafting." arXiv:2503.00784, 2025.
- Jonathan Mamou et al. "Dynamic Speculation Lookahead Accelerates Speculative Decoding of Large Language Models." arXiv:2405.04304, 2024.
- Xupeng Miao et al. "SpecInfer: Accelerating Generative Large Language Model Serving with Tree-based Speculative Inference and Verification." arXiv:2305.09781, 2023.
- MLX documentation: "Environment Variables", "Numerical Precision" and "Custom Metal Kernels". https://ml-explore.github.io/mlx/build/html/
- MLX issue #4595, "Option to opt out of TF32 for fp32 conv: small-kD conv3d is ~5000× less precise on M5 since 0.32", 29 Sep 2026. https://github.com/ml-explore/mlx/issues/4595
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Gabriele Oliaro et al. "SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications." arXiv:2411.04975, 2024.
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Apoorv Saxena. "Prompt Lookup Decoding" (software), Nov 2023. https://github.com/apoorvumang/prompt-lookup-decoding
- Ruslan Svirschevski et al. "SpecExec: Massively Parallel Speculative Decoding for Interactive LLM Inference on Consumer Devices." arXiv:2406.02532, 2024.
- Wenfeng Wang et al. "Specter: Fast MoE Inference via Quantization-Enabled Multi-Step Speculative Offloading." arXiv:2511.14102, 2025.
- Jinhui Wei et al. "Ghidorah: Fast LLM Inference on Edge with Speculative Decoding and Hetero-Core Parallelism." arXiv:2505.23219, 2025.
- Heming Xia et al. "ToolSpec: Accelerating Tool Calling via Schema-Aware and Retrieval-Augmented Speculative Decoding." arXiv:2604.13519, 2026.
- Jiebin Zhang et al. "DFlare: Scaling Up Draft Capacity for Block Diffusion Speculative Decoding." arXiv:2606.02091, 2026.
- Weilin Zhao et al. "FR-Spec: Accelerating Large-Vocabulary Language Models via Frequency-Ranked Speculative Sampling." arXiv:2502.14856, 2025.

Companion papers:

- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [KERNELS] Kamen Levi. Batch-invariant kernels on Apple's M5 GPU and its matrix units. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.